

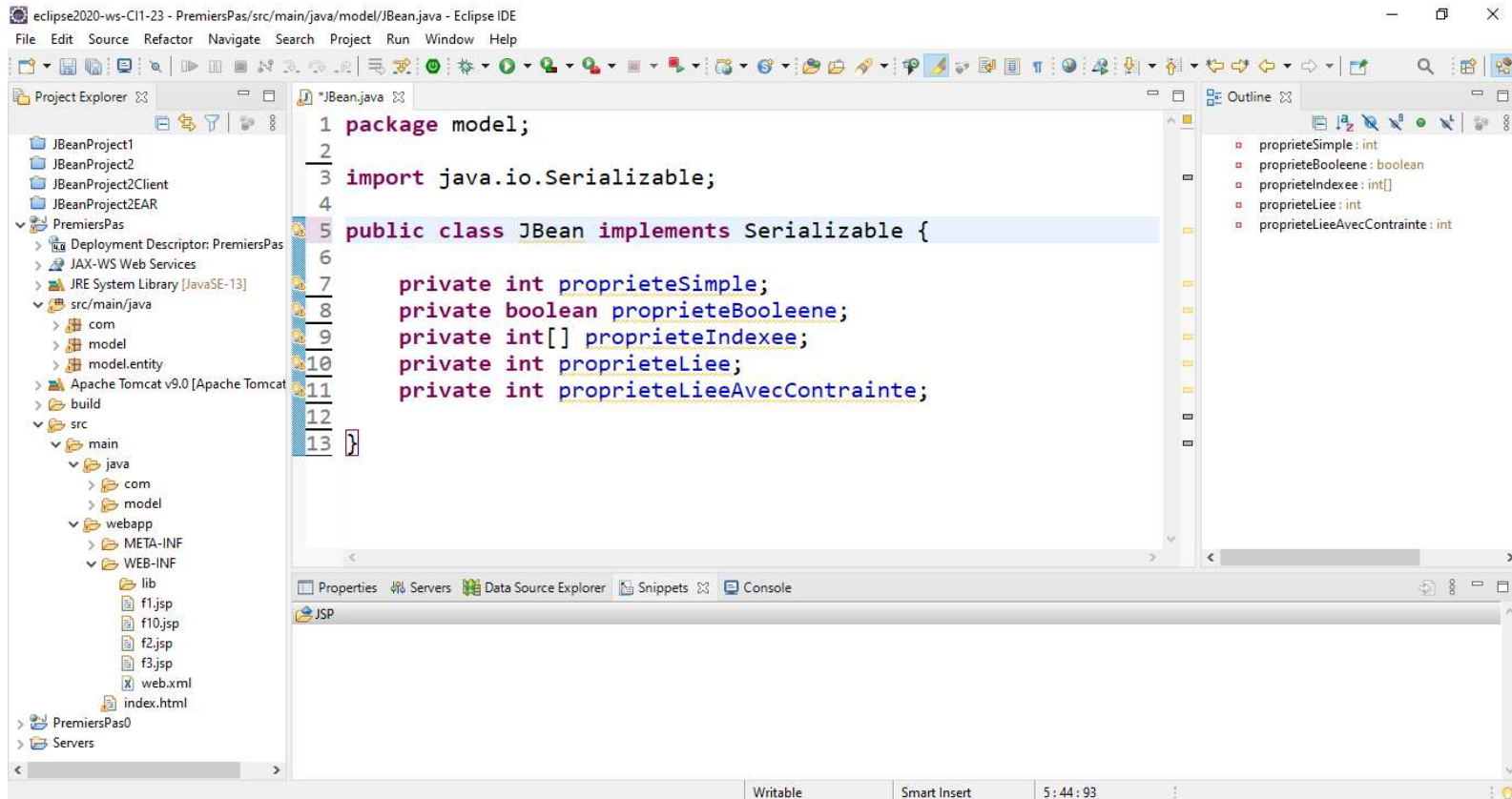
Tutoriel 4

PREMIER JAVA BEAN

Création d'un JBean

- Reprendre le même projet web dynamique du tutoriel 3 ou refaire les mêmes étapes
- Sous « src/main/java » créer un nouveau package « model »
- Créer une nouvelle classe sous « model » :
 - Hérite de l'interface « Serializable »
 - Possède 5 propriétés :
 - proprieteSimple : entier
 - proprieteBooleene : logique
 - proprieteIndexee : tableau d'entiers
 - proprieteLieu : entier
 - proprieteLieuAvecContrainte : entier

JBean et ses propriétés



Accesseurs de proprieteSimple

- Ajouter les accesseurs de la propriété simple en lecture (get) et écriture (set). Les deux accesseurs gèrent l'accès concurrent (synchronized)

```
12  private int proprieteSimple;  
13  //proprieteSimple accessible en lecture et ecriture  
14  //gestion de l'accès concurant par le mot clé synchronized  
15  public synchronized int getProprieteSimple() {  
16      return proprieteSimple;  
17  }  
18  public synchronized void setProprieteSimple(int proprieteSimple) {  
19      this.proprieteSimple = proprieteSimple;  
20  }
```

Accesseurs de proprieteBoolean

- Ajouter les accesseurs de la propriété logique en lecture (get ou is) et écriture (set).

```
23 private boolean proprieteBooleene;  
24 //proprieteBooleene accessible en lecture et ecriture  
25 public boolean isProprieteBooleene() {  
26     return proprieteBooleene;  
27 }  
28 public void setProprieteBooleene(boolean proprieteBooleene) {  
29     this.proprieteBooleene = proprieteBooleene;  
30 }
```

Accesseurs de proprieteIndexee

- Ajouter les accesseurs de la propriété indexée
 - 2 accesseurs non indexés :
 - en lecture : get() : tableau d'entier
 - en écriture : set(tableau d'entier)
 - 2 accesseurs indexés :
 - en lecture : get(entier) : entier
 - en écriture : set(entier, entier)

```
32 private int[] proprieteIndexee;  
33 //proprieteIndexee accessible en lecture et ecriture par 4 accesseurs  
34 //2 indexés et 2 non indexés  
35 public int getProprieteIndexee(int i) {  
36     return proprieteIndexee[i];  
37 }  
38 public void setProprieteIndexee(int i, int proprieteIndexee) {  
39     this.proprieteIndexee[i] = proprieteIndexee;  
40 }  
41 public int[] getProprieteIndexee() {  
42     return proprieteIndexee;  
43 }  
44 public void setProprieteIndexee(int[] proprieteIndexee) {  
45     this.proprieteIndexee = proprieteIndexee;  
46 }
```


Accesseurs de proprieteLiee

- Ajouter un écouteur pour la propriété liée
- Ajouter les accesseurs de la propriété liée en lecture et écriture
- Ajouter les méthodes d'ajout et de suppression de l'écouteur au JBean

```
48 private int proprieteLiee;  
49 PropertyChangeSupport changeSupport;  
50 //proprieteLiee accessible en lecture et ecriture  
51 //c'est l'accesseur en ecriture qui garde trace des modifications  
52 //d'où l'usage d'une instance de PropertyChangeSupport  
53 public synchronized int getProprieteLiee() {  
54     return proprieteLiee;  
55 }  
56 public synchronized void setProprieteLiee(int proprieteLiee) {  
57     int oldProprieteLiee = this.proprieteLiee;  
58     this.proprieteLiee = proprieteLiee;  
59     changeSupport.firePropertyChange("proprieteLiee", oldProprieteLiee, proprieteLiee);  
60 }  
61 public synchronized void addPropertyChangeListener(PropertyChangeListener listener) {  
62     changeSupport.addPropertyChangeListener(listener);  
63 }  
64 public synchronized void removePropertyChangeListener(PropertyChangeListener listener) {  
65     changeSupport.removePropertyChangeListener(listener);  
66 }
```

Accesseurs de proprieteLieeeAvecContrainte

- Ajouter un écouteur pour la vérification de modification pour la propriété liée avec contrainte.
- Ajouter les accesseurs de la propriété liée en lecture et écriture.
- Ajouter les méthodes d'ajout et de suppression de l'écouteur de vérification au JBean


```

68 private int proprieteLieceAvecContrainte;
69 private int oldValeur;
70 private VetoableChangeSupport vetoableSupport;
71 //proprieteLieceAvecContrainte accessible en lecture et ecriture
72 //c'est l'accesseur en ecriture qui garde trace des modifications
73 //et qui verifie si la modification est acceptable ou non
74 //d'où l'usage d'une instance de PropertyChangeSupport
75 //et d'une instance de VetoableChangeSupport
76 public synchronized int getProprieteLieceAvecContrainte() {
77     return proprieteLieceAvecContrainte;
78 }
79 public synchronized void setProprieteLieceAvecContrainte(int proprieteLieceAvecContrainte) {
80     oldValeur = this.proprieteLieceAvecContrainte;
81     this.proprieteLieceAvecContrainte = proprieteLieceAvecContrainte;
82
83     try {
84         vetoableSupport.fireVetoableChange("proprieteLieceAvecContrainte",oldValeur, this.proprieteLieceAvecContrainte);
85     } catch(PropertyVetoException e) {
86         System.out.println("JBean, un veto est emis : "+e.getMessage());
87         this.proprieteLieceAvecContrainte = oldValeur;
88     }
89     if (this.proprieteLieceAvecContrainte != oldValeur ) {
90         changeSupport.firePropertyChange("proprieteLieceAvecContrainte",oldValeur,this.proprieteLieceAvecContrainte);
91     }
92 }
93 public synchronized void addVetoableChangeListener(VetoableChangeListener listener) {
94     vetoableSupport.addVetoableChangeListener(listener);
95 }
96 public synchronized void removeVetoableChangeListener(VetoableChangeListener listener) {
97     vetoableSupport.removeVetoableChangeListener(listener);
98 }

```

Ajouter les constructeurs

- Ajouter au JBean 2 constructeurs :
 - Un constructeur par défaut
 - Un constructeur avec paramètres

```
99 public JBean() {
100     this.proprieteSimple = 1;
101     this.proprieteBooleene = true;
102     this.proprieteIndexee = new int[10];
103     this.proprieteLiece = 2;
104     this.proprieteLieceAvecContrainte = 3;
105
106     this.changeSupport = new PropertyChangeSupport(this);
107     this.vetoableSupport = new VetoableChangeSupport(this);
108 }
109 public JBean(int proprieteSimple, boolean proprieteBooleene, int[] proprieteIndexee, int proprieteLiece,
110             int proprieteLieceAvecContrainte) {
111     this.proprieteSimple = proprieteSimple;
112     this.proprieteBooleene = proprieteBooleene;
113     this.proprieteIndexee = proprieteIndexee;
114     this.proprieteLiece = proprieteLiece;
115     this.proprieteLieceAvecContrainte = proprieteLieceAvecContrainte;
116
117     this.changeSupport = new PropertyChangeSupport(this);
118     this.vetoableSupport = new VetoableChangeSupport(this);
119 }
```

Test du JBean

- Dans une autre classe, insérer un main qui implémente une instance du JBean

```
7
8 public class TestJBean {
9
10     public static void main(String[] args) {
11         //Création d'une instance du JB
12         JBean monBean = new JBean();
13
14     }
15 }
16
```

Test des différentes propriétés

CODE

```
//Test de la propriété simple  
  
System.out.println("Valeur de la  
propriété simple = " +  
monBean.getProprieteSimple());  
  
monBean.setProprieteSimple(10);  
  
    System.out.println("Valeur de  
la propriété simple = " +  
monBean.getProprieteSimple());
```

RÉSULTAT

Valeur de la propriété simple = 1

Valeur de la propriété simple = 10

Test des différentes propriétés

CODE

```
//Test de la propriété booleene  
  
System.out.println("Valeur de la  
propriété booleene = " +  
monBean.isProprieteBooleene());  
  
monBean.setProprieteBooleene(fal  
se);  
  
System.out.println("Valeur de la  
propriété booleene = " +  
monBean.isProprieteBooleene());
```

RÉSULTAT

Valeur de la propriété booleene = true

Valeur de la propriété booleene = false

Test des différentes propriétés

CODE

```
//Test de la propriété indexee avec les
accèsseurs non indexés

int tab[] = {00,10,20,30,40,50,60,70,80,90};

System.out.println("Contenu de la propriété
indexee : ");

for (int val : monBean.getProprieteIndexee())
    System.out.print(val+"\t");

monBean.setProprieteIndexee(tab);

System.out.println("\nContenu de la
propriété indexee : ");

for (int val : monBean.getProprieteIndexee())
    System.out.print(val+"\t");
```

RÉSULTAT

Contenu de la propriété indexee :

0	0	0	0
	0	0	0
	0	0	0

Contenu de la propriété indexee :

0	10	20	30
	40	50	60
	70	80	90

Test des différentes propriétés

CODE

```
//Test de la propriété indexee avec les accesseurs
indexés

System.out.println("\nContenu de la propriété
indexee : ");

for(int i=0;
i<monBean.getProprieteIndexee().length; i++)

        System.out.print(monBean.getProprie
teIndexee(i)+"\t");

for(int i=0;
i<monBean.getProprieteIndexee().length; i++)

        monBean.setProprieteIndexee(i,tab[i]
*2);

System.out.println("\nContenu de la propriété
indexee : ");

for(int i=0;
i<monBean.getProprieteIndexee().length; i++)

        System.out.print(monBean.getProprie
teIndexee(i)+"\t");
```

RÉSULTAT

Contenu de la propriété indexee :

0	10	20	30
	40	50	60
	70	80	90

Contenu de la propriété indexee :

0	20	40	60
	80	100	120
	140	160	180

Test des différentes propriétés

CODE

```
//Test de la propriété liée  
monBean.addPropertyChangeListener(  
    new PropertyChangeListener() {  
        public void propertyChange(PropertyChangeEvent  
event) { System.out.println("La propriété change de "+  
event.getOldValue()+" à "+ event.getNewValue());  
        }  
    }  
);  
  
System.out.println("\nValeur de la propriété liée = " +  
monBean.getProprieteLiee());  
  
monBean.setProprieteLiee(90);  
  
System.out.println("Valeur de la propriété liée = " +  
monBean.getProprieteLiee());
```

RÉSULTAT

Valeur de la propriété liée = 2

La propriété change de 2 à 90

Valeur de la propriété liée = 90

Test des différentes propriétés

CODE

```
//Test de la propriété liée avec contrainte

monBean.addVetoableChangeListener(

    new VetoableChangeListener() {

        public void
        vetoableChange(PropertyChangeEvent event)
        throws PropertyVetoException {

            System.out.println("vetoableChange :
nouvelle valeur = " + event.getNewValue());

            if(
((Integer)event.getNewValue()).intValue() > 100 )

                throw new
PropertyVetoException("Nouvelle valeur
superieure à 100",event);

        }

    });
```

SUITE DU CODE

```
System.out.println("Valeur de la propriété liée
avec contrainte = " +
monBean.getProprieteLieeAvecContrainte());

    monBean.setProprieteLieeAvecContrainte(10);

    System.out.println("Valeur de la propriété liée
avec contrainte = " +
monBean.getProprieteLieeAvecContrainte());

monBean.setProprieteLieeAvecContrainte(200);

    System.out.println("Valeur de la propriété liée
avec contrainte = " +
monBean.getProprieteLieeAvecContrainte());
```

RÉSULTAT

Valeur de la propriété liée avec contrainte = 3

La propriété change de 3 à 10

Valeur de la propriété liée avec contrainte = 10

La propriété change de 10 à 200

Valeur de la propriété liée avec contrainte = 200